Medición de métricas en bases documentales

MongoDB frente a PostgreSQL JSONB bajo contención

Ingeniería en Tecnologías de la Información e Innovación Digital

IT2iD

Universidad Politécnica de Chiapas · IT2iD

2026

Pregunta guía

¿Qué hay que medir, cómo se ejecuta y qué evidencia entregar para que un estudio posterior (y un revisor) puedan reproducir y cuestionar el resultado?



Al salir

Entender la matriz Mongo/Postgres × perfiles × Q1/Q2 × concurrencia, y el CSV/reporte que alimenta un estudio.

Actividad

Comparar **MongoDB** (documentos) y **PostgreSQL JSONB** al ejecutar agregaciones JSON anidadas, con límites estrictos de CPU y memoria (Docker cgroups v2).

- Dataset: 500 000 órdenes con $\sim$3 niveles de anidamiento.
- Dos cargas: Q1 (unwind + group) y Q2 (sort + Top-K).
- Tres perfiles: baseline / media / severa.
- Tres concurrencias: 1, 10, 50.

No es "probar si arranca"

Cada corrida produce **métricas numeradas** (latencia, QPS, throttling, RSS, spill, OOM) en un CSV homogéneo.

Preguntas que responden las métricas

M1 ¿Cómo cambian P95/P99 al limitar CPU y memoria?
M2 ¿Qué motor gestiona mejor buffers antes de spill o OOM?
M3 ¿Cuál es la huella de CPU/memoria por cada 1 000 documentos?

Para el equipo

Guían qué celdas priorizar y qué contraste reportar.

Para el estudio

Son las mismas preguntas que un revisor pedirá justificar con datos.

Perfiles de recursos (ejemplo operativo)

Qué anotar siempre

`profile`, límites Docker reales, versión del motor y si hubo spill/OOM. Sin eso, la celda no sirve para el estudio.

Ejemplo Q1: saturación de CPU

Desanidar `items`, filtrar `price > 50`, agrupar por país y categoría.

MongoDB (idea)

```
$unwind items
$match price > 50
$group by country,
         category
```

PostgreSQL (idea)

```
jsonb_array_elements
 + WHERE price > 50
 + GROUP BY country,
             category
```

Métrica a mirar

Throughput y P95/P99; en severa, tiempo CFS throttled.

Ejemplo Q2: estrés de buffer (Top-K)

Filtrar completadas, ordenar por `final_total`, Top-1000.

- Mongo: `allowDiskUse false/true`.
- Postgres: `work_mem` 4 MB / 64 MB.

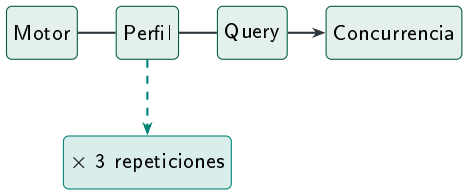
Ejemplo de lectura (ficticio, formato)

Perfil C, concurrencia 50, `allowDiskUse:false` ⇒ spill o fallo.
Misma celda con disco permitido ⇒ termina, pero baja QPS.

Para el revisor

Hay que reportar **ambas** variantes: sin spill y con spill (o OOM).

Una celda de la matriz (qué es "una medición")



- Matriz: $2 \times 3 \times 2 \times 3 = 36$ celdas; cada una $\times$ 3 repeticiones.
- Ciclo: flush caché $\rightarrow$ warmup 60 s $\rightarrow$ medición 300 s $\rightarrow$ cooldown 60 s.

Producto mínimo por celda

Una fila CSV por repetición + nota de estado (`ok/spill/oom/error`).

Ejemplo de fila CSV (formato esperado)

Sin columnas inventadas ni renombradas: el estudio posterior concatena todos los CSV.

Código / scripts

- `docker-compose` parametrizado
- generador 500k
- runner (k6 o Python)
- `queries/` Mongo + SQL
- `run_benchmarks.sh`

Evidencia

- CSV con el esquema fijo
- Capturas o logs de spill/OOM
- Reporte de métricas (7 secciones del playbook)

Criterio de utilidad

Si otro equipo no puede reejecutar o un revisor no puede cuestionar P99/throttling con sus datos, la entrega está incompleta.

Qué preguntará un revisor (anticípense)

1. ¿El baseline es comparable (mismo dataset, mismas queries)?
2. ¿Las latencias son P50/P95/P99 o solo promedios?
3. ¿Hubo throttling CFS? ¿Cuántos segundos?
4. ¿Spill a disco o OOM? ¿En qué perfil y concurrencia?
5. ¿Cuántas repeticiones y cómo se resume (media / mediana)?
6. ¿El plan de ejecución respalda el cuello de botella?

Su reporte debe contestarlas con tablas/figuras

No con opiniones: con filas del CSV y un plan `explain` / `EXPLAIN ANALYZE`.

Puente

El PDF `playbook.pdf` es la especificación; estas diapos fijan el contrato de entrega.

- Miden bajo contención, no solo “en máquina libre”.
- Cada celda → filas CSV + estado + (si aplica) spill/OOM.
- El entregable alimenta un estudio y resiste preguntas de revisor.

Siguiente paso en laboratorio

Levantar perfiles A/B/C, correr una celda piloto (1 motor $\times$ 1 query $\times$ 1 concurrencia) y validar el CSV antes de la matriz completa.